

软件工程基础第 7 章——详细设计

详细学习笔记

中南大学计算机学院 任胜兵

2026 年 5 月 28 日

目录

1	详细设计的目标与任务	2
1.1	概述	2
1.2	详细设计的目标	2
1.3	详细设计的任务	2
2	详细设计图形描述工具	3
2.1	概述	3
2.2	程序流程图 (Program Flowchart)	3
2.3	盒图 / N-S 图 (Box Diagram)	4
2.4	问题分析图 (PAD / Problem Analysis Diagram)	5
2.5	协作图 (Collaboration Diagram)	7
3	Jackson 程序设计方法 (JSP)	8
3.1	概述	8
3.2	Jackson 图 (Jackson Diagram)	8
3.3	Jackson 伪代码	9
3.4	JSP 方法的设计步骤	10
3.5	完整示例——装修公司构件周报表系统	10
4	Warnier 程序设计方法 (LCP)	12
4.1	概述	12
4.2	Warnier 图 (Warnier Diagram)	13
4.3	Warnier 方法的设计步骤	13
4.4	完整示例——报表生成系统	13
4.5	JSP 与 LCP 方法对比	14

5	面向对象程序的详细设计	15
5.1	概述	15
5.2	面向对象程序的三大特性	15
5.3	面向对象详细设计的原则	17
6	程序规格说明文档及复审	18
6.1	程序规格说明	18
6.2	程序规格说明复审	18
7	小结	19

1 详细设计的目标与任务

1.1 概述

[p.1-2]

详细设计（Detailed Design）是软件开发生命周期中承上启下的关键阶段。概要设计确定了软件系统的总体结构（模块划分、接口定义），而详细设计则对概要设计的结果进一步细化，给出目标系统的精确描述，以便在编码阶段直接翻译成计算机上能够运行的程序代码。

在详细设计阶段，结构化设计（Structured Design）和面向对象设计（Object-Oriented Design）没有本质区别——两者的核心任务都是确定算法（Algorithm）和数据结构（Data Structure）。详细设计的结果称为**软件详细规格说明**（Software Detailed Specification），相当于建筑工程中的施工图纸，与编程思想、方法和风格密切相关。

1.2 详细设计的目标

[p.3]

详细设计阶段具体地设计所要求的系统，得出新系统的**软件详细规格说明**。同时要求设计出的规格说明**简明易懂**，便于下一阶段用某种程序设计语言在计算机上实现。

核心要点：如何高质量地完成详细设计是提高软件质量的关键。详细设计规格说明的质量直接决定了后续编码阶段的效率以及最终软件产品的可维护性。

1.3 详细设计的任务

[p.4]

详细设计阶段需要完成以下六项主要任务：

1. **算法过程的设计**（Algorithm Design）——确定每个模块内部的执行逻辑和计算步骤，包括控制流程、处理顺序、条件判断、循环结构等。
2. **数据结构的设计**（Data Structure Design）——为每个模块确定所需的数据组织方式，包括数组、链表、树、栈、队列、哈希表等数据结构的选择和定义。
3. **数据库物理设计**（Physical Database Design）——将逻辑数据模型转化为特定数据库管理系统（DBMS）上的物理存储结构，涉及索引策略、分区方案、存储参数等。
4. **信息编码设计**（Information Coding Design）——确定系统中各类信息的编码规则和表示方式，如错误码体系、状态码定义、数据字典编码等。
5. **测试用例的设计**（Test Case Design）——基于详细设计规格说明，提前设计测试用例（白盒测试为主），包括输入数据、预期输出、执行路径覆盖等。
6. **编写详细设计说明书**（Detailed Design Document）——将以上各项设计结果整理成文档，作为编码阶段的直接依据。

易错点：很多项目在详细设计阶段忽略测试用例设计，等到编码完成才开始考虑测试，这会降低测试质量和效率。

2 详细设计图形描述工具

2.1 概述

[p.5]

详细设计中用于过程设计的图形工具主要包括四类：

- (1) **程序流程图**（Program Flowchart）——最经典的过程设计表示方法
- (2) **盒图**（Box Diagram / N-S 图 / Nassi-Shneiderman Diagram）——强制结构化设计
- (3) **问题分析图**（PAD / Problem Analysis Diagram）——树形层次化表示
- (4) **协作图**（Collaboration Diagram）——面向对象的对象交互表示

除此之外，**IPO 图**（Input-Process-Output）、**判定表**（Decision Table）、**判定树**（Decision Tree）以及**伪代码语言**（Pseudocode）等也可用于过程设计的描述。

设计建议：每种工具都有各自的优缺点，在实际设计中可以针对不同情况选用，甚至可以同时采用多种工具来描述详细设计的结果，以达到最佳表达效果。

2.2 程序流程图（Program Flowchart）

[p.6-7]

基本概念

程序流程图是最早出现、应用最广泛的算法描述工具。它使用标准化的图形符号来表示程序的控制流程：

- **矩形框**（Rectangle）：表示处理步骤或计算操作
- **菱形框**（Diamond / Rhombus）：表示条件判断和分支选择
- **带箭头的连线**（Arrow lines）：表示控制流的转移方向
- **椭圆形/圆角矩形**：表示开始和结束
- **平行四边形**（Parallelogram）：表示输入/输出操作

优缺点分析

主要优点：

- **直观性强：**图形化表示使程序逻辑一目了然，便于阅读理解
- **灵活性高：**可以通过箭头实现任意形式的控制转移，不受结构限制
- **易使用：**符号简单、规则少，便于初学者快速掌握
- **通用性好：**独立于任何特定程序设计语言

主要缺点：

- **不利于逐步求精：**程序流程图本质上并不是逐步求精 (Stepwise Refinement) 的好工具，它诱使程序设计人员过早考虑程序的控制流程细节，而不去考虑程序的全局结构 (Global Structure)。
- **可能导致非结构化设计：**用箭头表示控制流方向，可以实现控制流的任意转移。如果使用不当，非结构化的程序流程图可能非常难以理解，无法进行修改和维护。
- **不便于表示数据结构：**程序流程图主要关注控制流程，对数据结构的表达能力很弱。

应用场景：适合描述较小规模、逻辑不太复杂的模块算法。对于复杂的大型模块，建议结合其他工具（如盒图或 PAD 图）一起使用。

2.3 盒图 / N-S 图 (Box Diagram)

[p.8-9]

基本概念

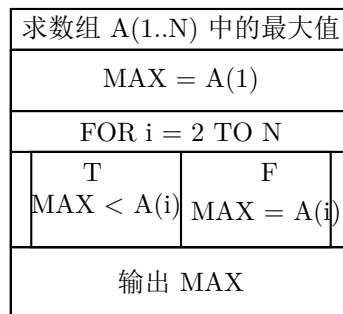
盒图 (Box Diagram) 由 Nassi 和 Shneiderman 于 1973 年提出，也称为 **N-S 图** (Nassi-Shneiderman Diagram)。盒图的核心设计理念是：**用盒状的图形结构代替程序流程图中带箭头的控制流线**，使得控制流的作用域被严格限定在结构化的盒子中。

盒图的基本控制结构

盒图使用以下基本控制结构来表示程序逻辑：

- (1) **顺序结构 (Sequence)：**多个处理框按从上到下的顺序垂直排列，表示依次执行。
- (2) **选择结构 (Selection / if-then-else)：**用带条件标记的分隔框表示二路或多路分支选择。
- (3) **循环结构 (Iteration / while-do / repeat-until)：**用带有循环条件标记的框表示重复执行。
- (4) **调用结构 (Subroutine Call)：**用特殊标记的框表示对子程序或模块的调用。

盒图示例——求最大值



优缺点分析

主要优点：

- **强制结构化：**盒图强迫设计人员使用结构化程序设计技术，从根本上杜绝了非结构化的 GOTO 式控制转移，保证了设计质量。
- **控制作用域清晰：**从盒图上可以直观地看出某一特定控制结构（如循环、条件）的作用范围（Scope），为理解设计意图、编程实现、选择测试用例等带来了方便。
- **便于数据描述：**使用时可以附加一个描述数据结构的盒子，使得盒图更加适用于详细设计。

主要缺点：

- **修改困难：**盒图的修改比较麻烦，尤其是在嵌套层次较深时。
- **嵌套层次多时不易绘制：**当程序结构嵌套层次较多时，盒子的尺寸会变得很大，也不太容易在纸张或屏幕上绘制。
- **普及度不足：**由于以上原因，盒图的使用至今仍不很流行。

2.4 问题分析图（PAD / Problem Analysis Diagram）

[p.10-11]

基本概念

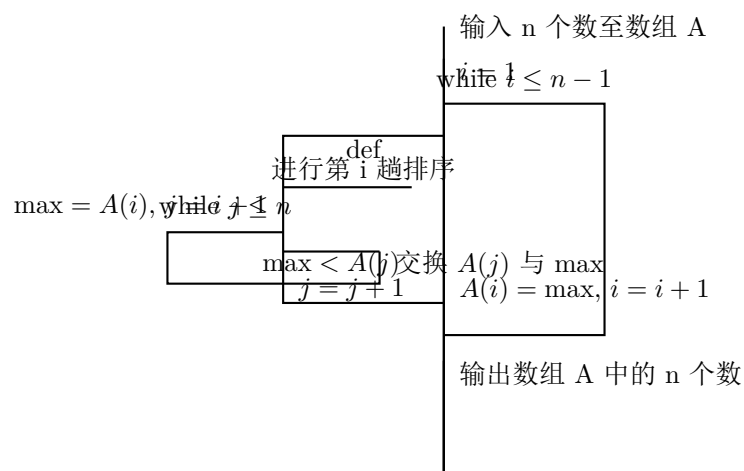
问题分析图（Problem Analysis Diagram，简称 **PAD**）由日本日立公司二村良彦等人于 1979 年提出。PAD 图采用**树形结构**（Tree Structure）来表示程序的控制逻辑，是一种支持结构化程序设计的图形工具。

PAD 图的基本结构

PAD 图使用以下三种基本结构：

- (1) 顺序结构：多个处理框水平依次排列，用水平线连接。
- (2) 选择结构：从条件框引出分支线，每条分支线上标记条件值。
- (3) 循环结构：用带有循环条件标记的纵向延伸框表示（while 型和 until 型）。
- (4) def 结构：用于定义子过程或函数调用。

PAD 图示例——冒泡排序



PAD 图的特点

- 强制结构化：与盒图一样，PAD 图也强制设计人员采用结构化程序设计技术，不会出现非结构化的控制流。
- 树形结构层次清晰：PAD 图采用树形结构表示程序逻辑，既克服了程序流程图不能清晰表现程序层次（Hierarchy）结构的缺点，又不同于盒图将处理约束在一个盒子里而使修改麻烦。
- 支持自动生成代码：PAD 图的树形结构为软件的自动生成提供了有力帮助——通过对 PAD 图的树形遍历（Tree Traversal），可以自动生成程序代码。
- 便于修改：树形结构使局部修改不影响整体结构，比盒图灵活。

对比总结：

- 程序流程图：灵活但可能非结构化，适合小型简单模块
- 盒图/N-S 图：强制结构化但修改困难，适合文档化展示
- PAD 图：强制结构化且易修改，支持自动代码生成，适合中大项目

2.5 协作图（Collaboration Diagram）

[p.12-13]

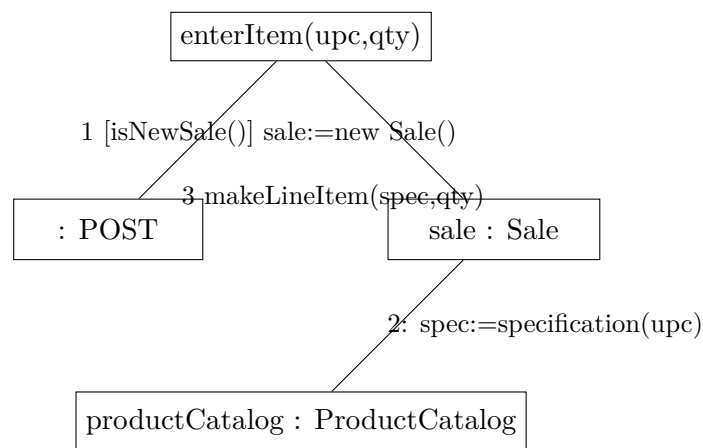
基本概念

协作图（Collaboration Diagram）是 **UML**（Unified Modeling Language）中的一种交互图（Interaction Diagram），用于表示对象之间的交互关系和消息传递。与顺序图（Sequence Diagram）并列为 UML 的两种交互图。

协作图的核心要素

- **对象（Object）**：用矩形框表示，框内标注对象名和类名（格式：**objectName : ClassName**）。
- **连接线（Link / Connection）**：表示对象之间的静态关联关系。
- **消息（Message）**：沿连接线标注的带箭头标记，表示对象间的方法调用。消息格式为：序号 方法名（参数）。
- **序号（Sequence Number）**：消息前的数字表示消息发送的顺序，用嵌套编号（如 1, 1.1, 1.2, 2）表示层次。

协作图示例——商品录入系统



协作图与顺序图的对比

[p.13]

协作图和顺序图（Sequence Diagram）都可用来表示对象之间的交互关系，两种形式之间可以相互转换。它们的区别和适用场景如下：

比较维度	协作图	顺序图
关注重点	对象之间的静态连接关系（结构组织）	消息传递的时间顺序（动态时序）
时间表达	用序号表示消息先后顺序，不强调时间轴	用垂直生命线清晰表示时间流逝，时间感强
空间效率	更节省绘图空间，适合对象较多时	对象多时水平展开，占用空间较大
结构清晰度	对象间的静态关系一目了然	静态关系不如协作图直观
适用场景	强调对象结构关系的设计讨论	强调消息时序的使用场景分析

表 1: 协作图与顺序图的对比

适用建议：在详细设计阶段，当需要重点描述对象间的静态关联和整体结构时优先使用协作图；当需要展现复杂交互的时间顺序时使用顺序图。

3 Jackson 程序设计方法（JSP）

3.1 概述

[p.14]

Jackson 结构化程序设计方法（Jackson Structured Programming，简称 **JSP**）由英国人 M. A. Jackson 首先提出，是一种**面向数据结构**（Data-Structure-Oriented）的结构化程序设计方法。

核心思想：JSP 方法的本质是”数据结构决定程序结构”——通过分析问题的输入数据结构和输出数据结构，找出两者之间的对应关系，按一定的映射规则将其映射成软件的过程描述，最后用 Jackson 伪代码表示。

适用领域：JSP 方法特别适用于数据处理类（Data Processing）系统，即输入数据和输出数据都有清晰结构的业务系统。

3.2 Jackson 图（Jackson Diagram）

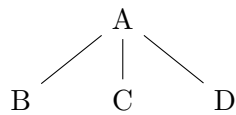
[p.15]

Jackson 图是 JSP 方法中用于表示数据结构和程序结构的图形化工具。它使用层次化的树形图来表示数据的组成关系。

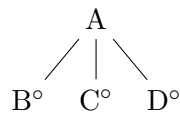
Jackson 图的三种基本结构

- (1) **顺序结构（Sequence）：**数据由多个成分按顺序组成，从上到下排列，连接线不加标记。在图中表示为 B、C、D 依次排列在 A 之下。

- (2) **选择结构 (Selection)**: 数据由多个可选成分之一组成。在选择关系的连接线上标记小圆圈 (°), 表示” 或” 的关系。B°、C°、D° 表示三者选一。
- (3) **循环结构 (Iteration / Repetition)**: 数据由某个成分重复出现组成。在循环关系的连接线上标记星号 (*), 表示” 重复”。B* 表示 B 出现 0 次或多次。



顺序 (Sequence)



选择 (Selection)



循环 (Iteration)

3.3 Jackson 伪代码

[p.16]

Jackson 伪代码是 Jackson 图的文本表示形式, 也是 JSP 方法的最终输出。

三种基本结构的伪代码表示

- (1) 顺序结构:

```

A seq
  B
  C
  D
end A
  
```

- (2) 选择结构:

```

A select condition S(j)
  B    or condition 1
  C    or condition 2
  D
end A
  
```

- (3) 循环结构:

```

A iter until (or while) condition I(j)
  B
end A
  
```

3.4 JSP 方法的设计步骤

[p.17]

Jackson 程序设计方法分为以下四个步骤：

- 步骤 1：分析并确定输入输出数据结构：**分析问题的输入数据和输出数据，确定各自的数据结构，并用 Jackson 图表示。
- 步骤 2：找出对应关系：**找出输入数据结构和输出数据结构中有对应关系的数据单元。所谓“对应关系”是指有直接的因果关系，在程序中可以或需要同时处理的单元。**关键规则：**对于重复出现的数据单元，必须满足“重复的次序和次数在输入与输出数据结构中都相同”，才可能有对应关系。
- 步骤 3：数据结构映射为程序结构：**按照映射规则将数据结构映射为程序结构图。程序结构图的层次与输入/输出数据结构图的层次对应。
- 步骤 4：列出操作和条件并分配：**列出完成程序结构图中各处理框功能的全部操作，以及有关的条件判断，并将它们分配到程序结构图的适当位置。
- 步骤 5：用 Jackson 伪代码写出过程性描述：**将程序结构图转换为 Jackson 伪代码。

3.5 完整示例——装修公司构件周报表系统

[p.18-23]

问题描述

某装修公司的仓库存放有多种装修构件（如内角线、九夹板等），每种构件的每次进货、出货都有相应的记录（包括构件编号、名称、数量、入/出库操作类型），存入构件记录表中。装修公司每周根据构件记录表打印一张**周报表**，报表的每行列出某种构件本周变化的数量。

输入：构件记录表（编号、名称、数量、操作类型）

输出：装修构件周报表（编号、变化量）

步骤 1：分析并确定输入输出数据结构

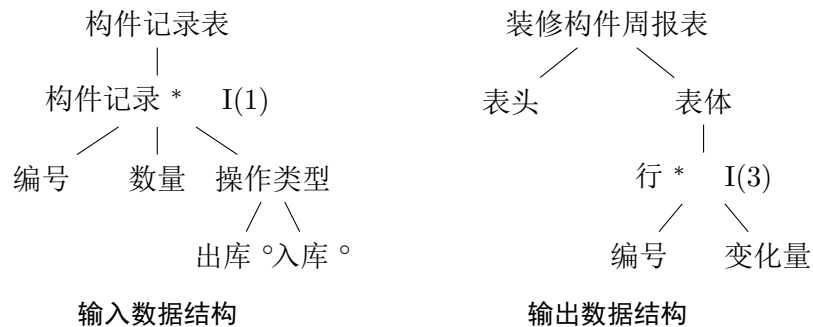
[p.19]

输入数据结构——构件记录表：

- 构件记录表由多个构件记录组成（循环 I(1)：while 构件记录表未空）
- 每个构件记录包含：编号、数量、操作类型
- 操作类型有出库（°）和入库（°）两种选择 S(2)

输出数据结构——装修构件周报表：

- 报表由表头和表体组成（顺序 S(2)）
- 表体由多个行组成（循环 I(3)：while 所有构件类型）
- 每行包含：编号、变化量



步骤 2：找出对应关系

[p.20]

输入和输出数据结构之间的对应关系中，关键观察是：构件记录表中的每条记录按编号分组后，与周报表中每个构件行的编号和变化量之间存在对应关系。

引入中间数据结构”构件组记录”来桥接输入和输出：

- I(1) 表示”while 所有构件类型”——对应输出中按构件类型分组
- I(2) 表示”while 某一构件类型（组）的所有构件记录”——对应同编号的多条记录
- S(3) 表示”是出库还是入库”——决定变化量的正负

步骤 3：数据结构映射为程序结构

[p.21]

映射规则（Mapping Rules）：

- 规则 1：** 对每对有对应关系的数据单元，按其在数据结构图中的层次，在程序结构图的相应层次画一个处理框。
- 规则 2：** 如果这对数据单元在输入和输出数据结构中处于不同的层次，则处理框在程序结构图中的层次与它们在数据结构图中层次较低的一方对应（高层在上，低层在下）。
- 规则 3：** 对输入数据结构中剩余的数据单元，根据其层次在程序结构图相应层次画处理框。
- 规则 4：** 对输出数据结构中剩余的数据单元，根据其层次在程序结构图相应层次画处理框。

步骤 4：列出并分配操作与条件

[p.22]

操作列表（共 17 项）：

- (1) 输入构件记录库文件名，(2) 打开构件记录库文件名
- (3) 对构件记录库文件按编号索引，(4) 创建报表文件
- (5) 生成表头字符串至报表文件，(6) 送换行符至报表文件
- (7) 变化量 $change = 0$ ，(8) ID 置为当前构件记录的编号
- (9) 读构件记录库文件的编号至 ID，(10) 读构件记录库文件的数量至 data
- (11) $change = change - data$ （出库），(12) $change = change + data$ （入库）
- (13) 指向构件记录库文件的下一条记录，(14) 生成编号 ID 字符串至报表文件
- (15) 生成变化量 $change$ 字符串至报表文件，(16) 关闭构件记录库文件
- (17) 关闭报表文件

条件定义：

- I(1)：构件记录库文件未到文件尾
- I(2)：当前构件记录的编号 $\neq$ ID（表示换了一个新的构件类型）
- S(3)：当前构件记录的操作类型是出库

步骤 5：Jackson 伪代码表示

[p.23]

生成装修构件周报表的伪代码以 **seq** 开始，先进行文件初始化（输入文件名、打开文件、索引、创建报表、生成表头），然后进入主循环（**iter while** 构件记录库文件未到文件尾），处理每个构件组记录生成表行，最后关闭文件。

4 Warnier 程序设计方法（LCP）

4.1 概述

[p.24]

Warnier 程序设计方法由法国人 J. D. Warnier 提出，全称为 **LCP**（Logical Construction of Programs，逻辑构造程序）方法。与 JSP 方法类似，Warnier 方法也是一种面向数据结构的程序设计方法。

Warnier 方法的设计流程：

- (1) 分析输入和输出数据结构，用 Warnier 图表示
- (2) 从数据结构（主要是输入数据结构，输出数据结构用于完善输入数据结构）导出程序结构，同样用 Warnier 图表示
- (3) 将程序结构改用程序流程图表示
- (4) 根据程序流程图写出程序的详细过程性描述

4.2 Warnier 图 (Warnier Diagram)

[p.25]

Warnier 图使用花括号层次嵌套的方式来表示数据的层次组成关系。它与 Jackson 图的不同在于：Warnier 图使用从左到右展开的花括号层次，而非树形结构。

Warnier 图的符号表示

- 花括号 {：表示层次分解，花括号后的项目是花括号前项目的子成分
- $\oplus$ (异或/排他)：表示选择关系，在多个选项之间二选一或多选一
- (n) 或数字标注：表示重复（循环）次数， (n) 表示出现 n 次

4.3 Warnier 方法的设计步骤

[p.26]

- 步骤 1：分析输入输出数据结构：**分析和确定问题的输入和输出数据结构，并用 Warnier 图来表示。
- 步骤 2：导出程序处理结构：**从数据结构（特别是输入数据结构）导出程序的处理层次结构，用 Warnier 图表示。
- 步骤 3：转为程序流程图：**将 Warnier 图表示的程序结构改用程序流程图表示。
- 步骤 4：写出详细过程性描述：**根据程序流程图写出程序的详细过程性描述。

4.4 完整示例——报表生成系统

[p.27-31]

问题描述

设计一个报表生成系统，该系统能读入包含各部门信息的输入文件，并生成按部门组织的汇总报表。

步骤 1——分析输入输出数据结构 [p.28]: 用 Warnier 图分别表示输入文件的层次结构（文件 → 部门记录 → 详细信息）和输出报表的层次结构（报表 → 表头/表体 → 部门行 → 汇总数据）。

步骤 2——导出程序处理层次结构 [p.29]: 从输入数据结构推导出程序的处理层次，包括文件处理层 → 部门处理层 → 记录处理层。

步骤 3——画出程序流程图 [p.30]: 将 Warnier 程序结构按照结构化编程的三种基本控制结构（顺序、选择、循环）转化为程序流程图。

步骤 4——写出详细过程性描述 [p.31]:

具体操作：

- (1) 自上而下给流程图每个处理框统一编号。
- (2) 列出每个处理框所需要的指令，冠以处理框的序号，并将指令分成如下五类：
 - **A 类：**输入和输入准备指令（Input & Input Preparation）
 - **B 类：**分支和分支准备指令（Branch & Branch Preparation）
 - **C 类：**计算指令（Computation）
 - **D 类：**子程序调用指令（Subroutine Call）
 - **E 类：**输出和输出准备指令（Output & Output Preparation）
- (3) 将”分类指令表”中的指令全部按处理框序号重新排序。序号相同则基本按”输入 → 处理 → 输出”的顺序排列，从而得到程序的详细过程性描述。

4.5 JSP 与 LCP 方法对比

比较维度	JSP (Jackson)	LCP (Warnier)
提出者	M. A. Jackson（英国）	J. D. Warnier（法国）
图形工具	Jackson 图（树形图）	Warnier 图（花括号层次图）
文本工具	Jackson 伪代码（Schematologic）	分类指令表
核心思想	数据结构映射为程序结构	逻辑构造程序
输出形式	伪代码（seq/select/iter）	分类指令排序表
中间表示	直接使用 Jackson 图	需先转为程序流程图

表 2: JSP 方法与 LCP 方法对比

5 面向对象程序的详细设计

5.1 概述

[p.32]

面向对象设计（Object-Oriented Design, OOD）在详细设计阶段主要完成对象的属性（Attribute）和方法（Method）的设计，称为面向对象程序的详细设计（Object-Oriented Detailed Design）。

面向对象程序的详细设计与结构化程序的详细设计有相似之处（都需要确定算法和数据结构），但也有其特殊之处，在设计时要认真考虑面向对象的特性才能充分发挥其优越性。

5.2 面向对象程序的三大特性

[p.33-35]

1. 封装性（Encapsulation）

[p.33]

定义：封装性是指类的封装机制使得数据和操纵数据的算法（函数或过程）紧密地捆绑在一起，从而将方法和数据的作用域（Scope）和可视性（Visibility）限制在软件系统的局部区域内。

封装的关键要素：

- **数据隐藏（Data Hiding）：**通过 `private` / `protected` 访问控制将内部数据保护起来
- **接口与实现分离（Interface vs. Implementation）：**公开的接口（`public` 方法）与私有的实现细节（`private` 成员）分离
- **访问控制（Access Control）：**C++ 中使用 `public`、`protected`、`private` 三级访问权限

封装的好处：

- **降低耦合：**修改类的内部实现不影响外部调用代码
- **提高安全性：**防止外部代码直接篡改对象内部数据
- **便于维护：**实现细节变更影响范围被限定在类内部

2. 继承性（Inheritance）

[p.34]

定义：在面向对象程序设计中，允许某个类继承其他类的成员函数或数据成员。被继承的类称为**基类**（Base Class）、**父类**（Parent Class）或**超类**（Superclass）；继承的类称为**派生类**（Derived Class）或**子类**（Subclass）。

继承的类型：

- **单继承** (Single Inheritance): 一个派生类只有一个基类
- **多重继承** (Multiple Inheritance): 一个派生类有多个基类 (C++ 支持, Java/C# 不支持类的多重继承)
- **访问权限继承**: `public` 继承、`protected` 继承、`private` 继承 (C++ 语法)

继承的好处:

- **代码复用** (Code Reuse): 子类自动获得父类的属性和方法
- **层次化建模** (Hierarchical Modeling): 自然表达“is-a”关系
- **扩展性** (Extensibility): 通过添加子类来扩展系统功能

C++ 继承语法示例:

```
class CDerivedClass : public CBaseClass {  
    // 派生类特有的成员  
};
```

3. 多态性 (Polymorphism)

[p.35]

定义: 多态性 (Polymorphism) 使得相关的类可有同名的函数, 这个同名的函数根据不同类的对象产生不同的结果。换言之, 不同类的对象可以具有相同的接口 (Interface), 这些相同的接口自然会呈现出不同的行为 (Behavior)。

多态性的类型:

- **编译时多态** (Compile-time Polymorphism): 通过函数重载 (Function Overloading) 和运算符重载 (Operator Overloading) 实现, 在编译时确定调用哪个函数。
- **运行时多态** (Run-time Polymorphism): 通过虚函数 (Virtual Function) 和动态绑定 (Dynamic Binding) 实现, 在运行时根据对象的实际类型确定调用哪个方法。

C++ 中多态性的实现——虚函数:

- 使用 `virtual` 关键字声明虚函数
- 基类指针或引用指向派生类对象时, 通过虚函数表 (vtable) 实现动态分派
- 可以编写通用代码处理未知类型的对象

C++ 虚函数示例:

```
class CDate {
protected:
    int year, month, day;
public:
    virtual void GetDateString(String80 &DateString);
    void DisplayDateString(void); // 将调用 GetDateString
};

// 在 CDate 的子类中如果重写了 GetDateString,
// 会影响 DisplayDateString 的行为（多态性）
```

5.3 面向对象详细设计的原则

[p.36]

1. 可复用性（Reusability）原则

- 保证方法的内聚性（Cohesion）：一个方法只做好一件事
- 减少方法的代码规模：方法不宜过长，便于理解和复用
- 保持方法对外接口的一致性（Consistency）：相似功能使用相似的方法签名
- 分离策略方法和实现方法（Strategy vs. Implementation）：策略（控制逻辑）方法与实现（具体执行）方法分开
- 方法应均匀覆盖数据：避免某些数据有过多的操作方法而其他数据缺乏操作
- 加强封装性：减少对外暴露的接口，提高内部实现的自由度
- 减少方法的耦合性（Coupling）：方法之间的依赖尽量少

2. 可扩展性（Extensibility）原则

- 封装数据：避免外部直接访问数据，通过方法接口访问
- 封装方法内部的数据结构：数据结构变更不影响调用者
- 避免情况分支语句：用多态替代 switch-case 或 if-else-if 链（策略模式思想）
- 区分公有方法和私有方法：公有方法是暴露给外部的契约，私有方法是内部实现细节

3. 健壮性（Robustness）原则

- 防止输入错误（围栏 / Fence）：对所有外部输入进行合法性校验
- 把握优化代码的时机：先保证正确性，再进行优化；避免过早优化
- 检查参数的合法性（Parameter Validation）：在方法入口处验证参数
- 选择适当的实现方法：在正确性和效率之间权衡

4. 协作性（Collaboration）原则

- 对类进行详细的文档化：清晰描述类的职责、接口、使用约束
- 把类打包成模块：将功能相关的类组织成包或模块
- 尽量使得代码容易理解：命名清晰、逻辑简洁、注释适度

6 程序规格说明文档及复审

6.1 程序规格说明

[p.37]

程序规格说明（Program Specification）又称详细设计说明（Detailed Design Description），与概要设计说明相比，程序规格说明着重描述模块的细节，包括：

- 算法过程：每个模块内部的详细算法描述（控制流、处理步骤）
- 数据结构：每个模块使用的数据组织方式（变量类型、结构定义、存储方案）
- 接口细节：模块的输入参数、输出参数、调用方式
- 异常处理：模块中各种异常情况的处理逻辑

简化处理：如果软件系统比较简单，则可将程序规格说明并入概要设计说明中，不必单独成文。

6.2 程序规格说明复审

[p.38]

程序规格说明的复审（Review）类似于概要设计说明的复审，但重点在于各个模块的具体设计上。

复审要点（Review Checklist）：

- （1）模块的详细设计能否满足其**功能（Functionality）**与**性能（Performance）**要求？

- (2) 详细设计描述是否**简单、清晰** (Simple & Clear)?
- (3) 详细设计选择的**算法和数据结构**是否合理?
- (4) 设计是否符合程序实现语言的特点 (如语言的范式、限制、最佳实践)?
- (5) 模块间的**接口**是否定义明确、一致?
- (6) 是否考虑了**异常情况**和边界条件?
- (7) 设计是否支持后续的**测试和维护**?

复审目的: 在编码之前发现详细设计中的问题, 避免将设计缺陷带入编码阶段, 从而降低修复成本。

7 小结

详细设计是进行软件系统开发的最后一个逻辑设计阶段, 其质量的好坏将直接影响到系统的编码实现。本章的核心要点总结如下:

1. **合理选择和正确使用设计工具:** 程序流程图 (直观灵活)、盒图/N-S 图 (强制结构化)、PAD 图 (树形层次 + 自动代码生成)、协作图 (对象交互), 各工具各有适用范围, 在实际项目中可以组合使用。
2. **深入理解和掌握设计方法:** Jackson 结构化程序设计方法 (JSP) 和 Warnier 逻辑构造程序方法 (LCP) 都基于”数据结构驱动程序设计”的理念, 适用于数据处理类系统。
3. **结构化与面向对象的共性:** 结构化程序的详细设计与面向对象程序的详细设计有许多共性——两者都需要确定算法和数据结构, 只是面向对象在此之上还强调封装、继承、多态三大特性以及复用性、扩展性、健壮性和协作性四大原则。
4. **文档与复审不可忽视:** 程序规格说明 (详细设计说明书) 是编码的直接依据, 必须通过正式复审确保质量, 避免设计缺陷进入编码阶段。

文档编制: 2026 年 5 月 28 日

来源课件: 软件工程基础第 7 章详细设计 (中南大学任胜兵)

共 39 页课件, 全部内容已覆盖